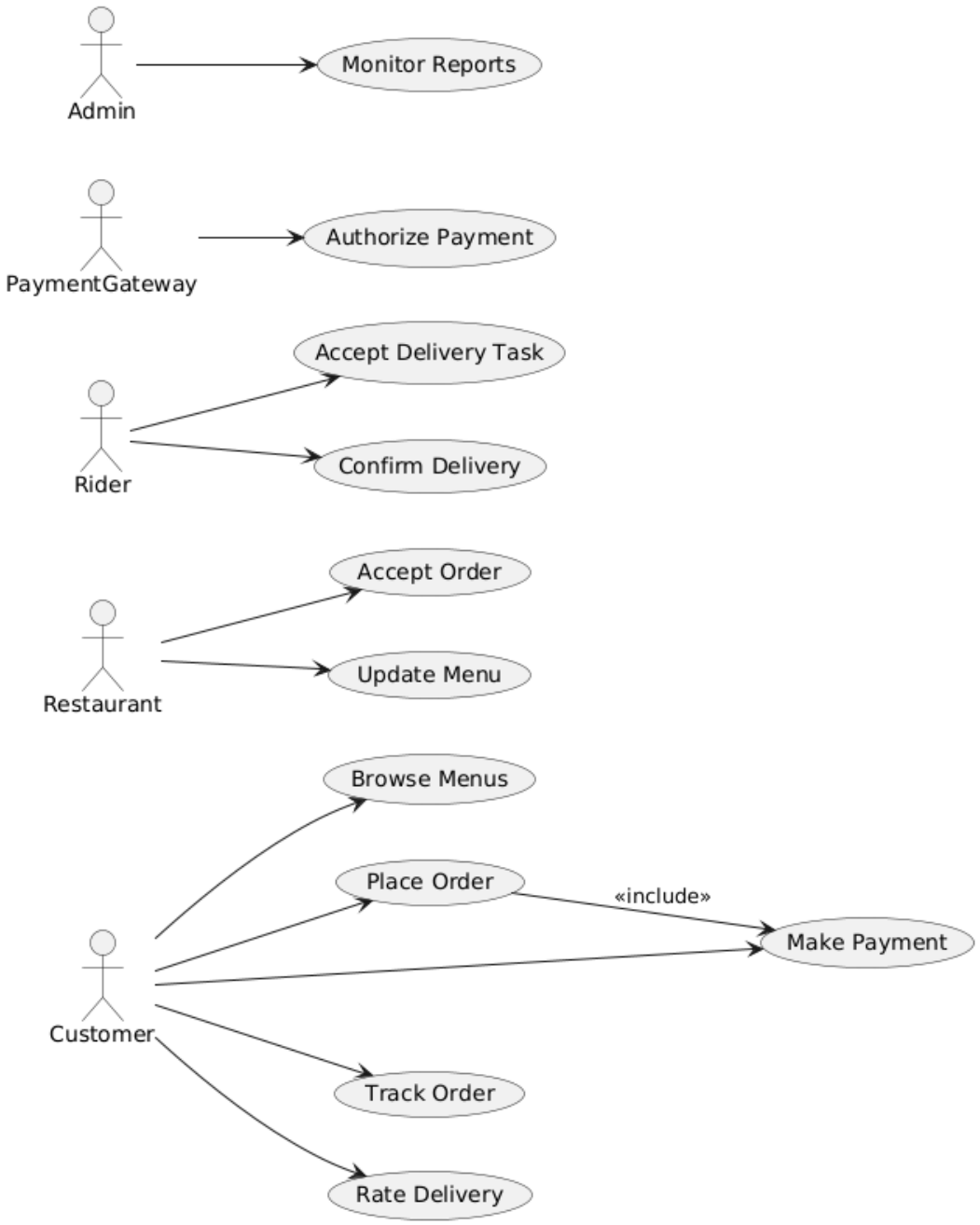
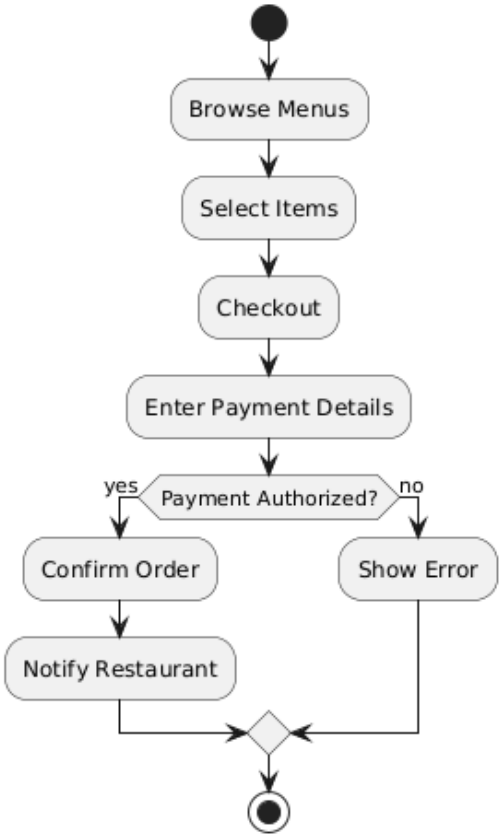
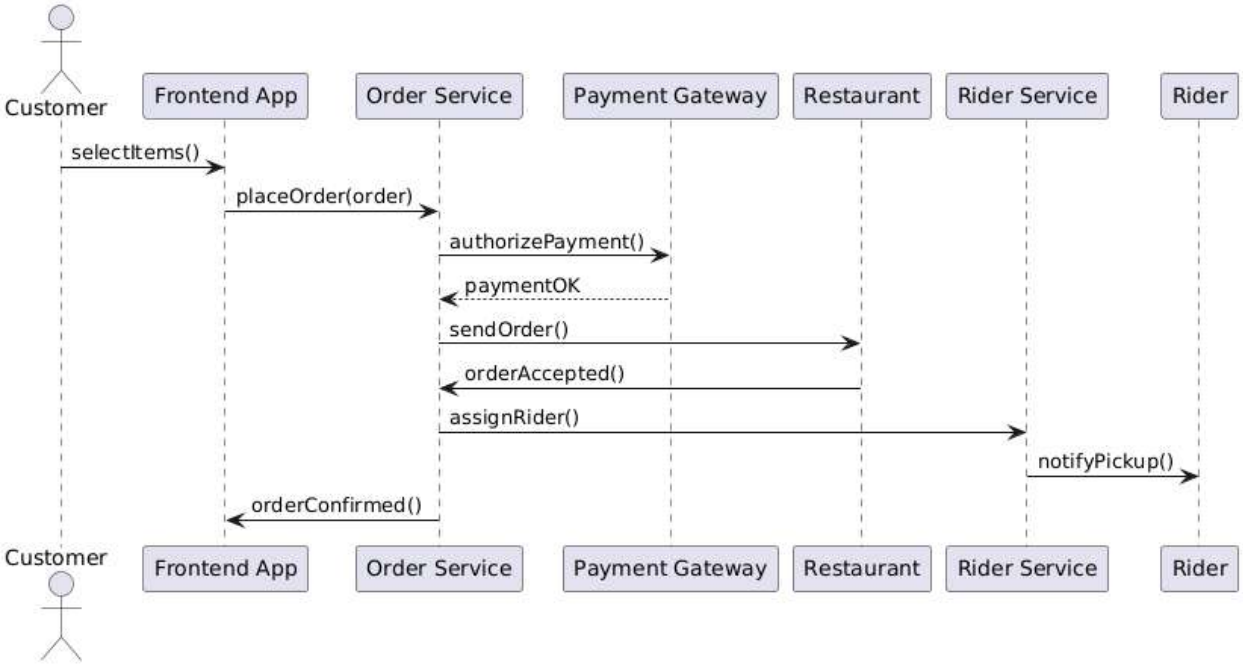
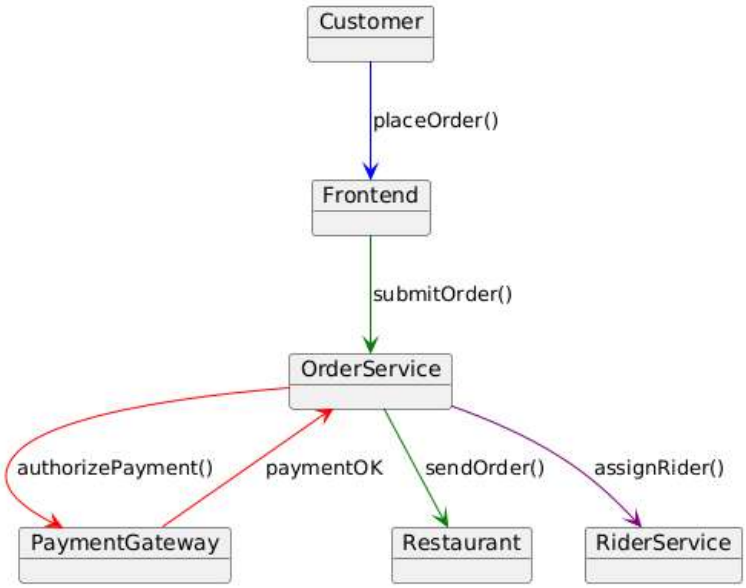
# SAMPLE 1

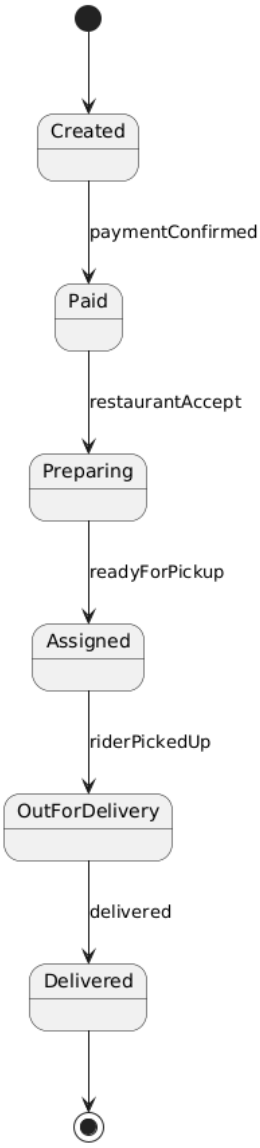
DeliverEase is an online marketplace for prepared food where Customers browse menus and place orders from multiple Restaurants. Restaurants receive orders, prepare meals, and Riders (couriers) deliver them. The platform handles order lifecycle, payments, real-time tracking, ratings, and dispute resolution. Admins and restaurant managers can manage menus and promotions.

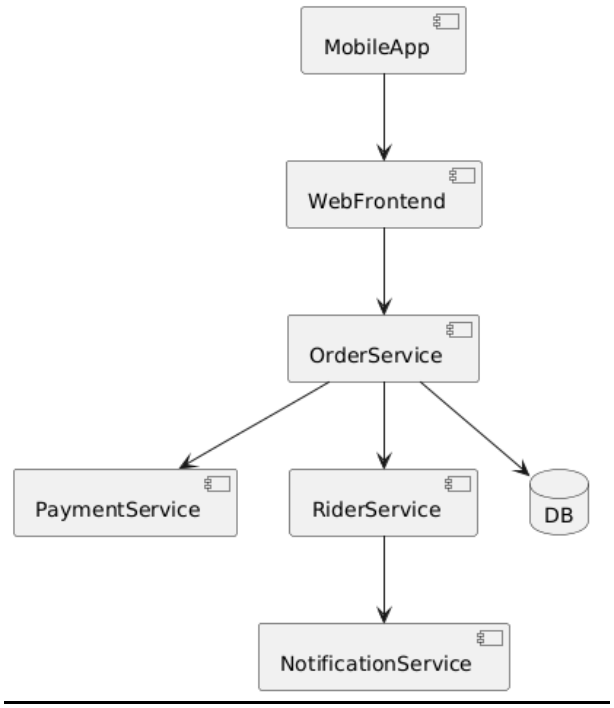


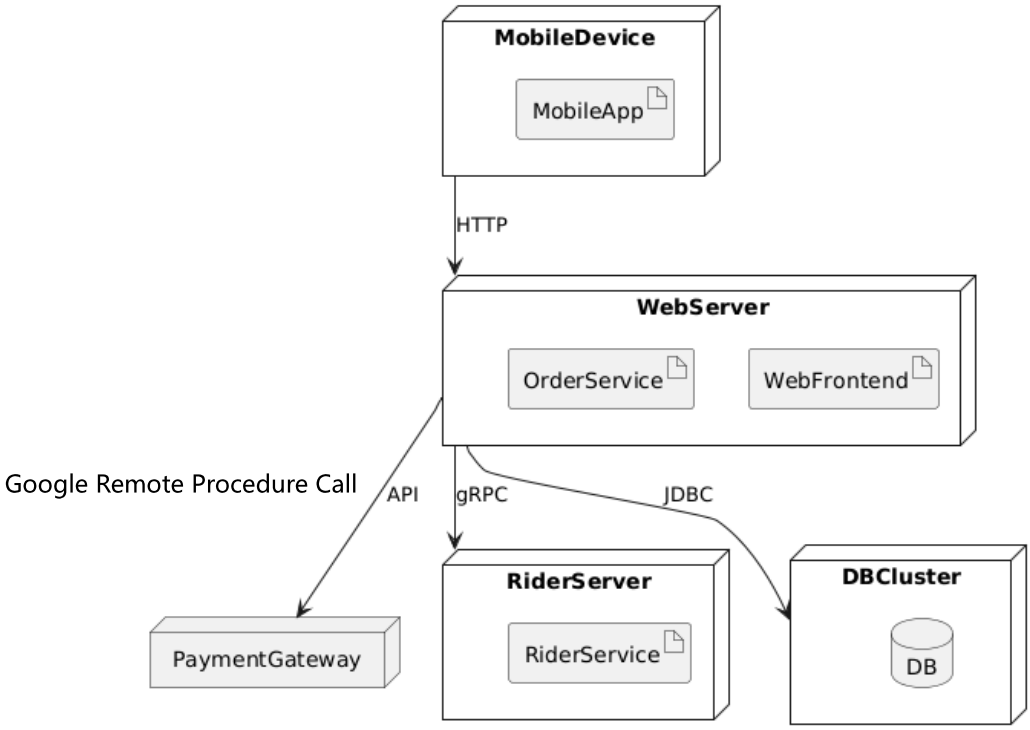

Browse Menus
Select Items
Checkout
Enter Payment Details
Payment Authorized?
yes
no
Confirm Order
Show Error
Notify Restaurant


Customer
Frontend App
Order Service
Payment Gateway
Restaurant
Rider Service
Rider
selectItems()
placeOrder(order)
authorizePayment()
paymentOK
sendOrder()
orderAccepted()
assignRider()
notifyPickup()
orderConfirmed()


Customer
placeOrder()
Frontend
submitOrder()
OrderService
authorizePayment()
paymentOK
sendOrder()
assignRider()
PaymentGateway
Restaurant
RiderService

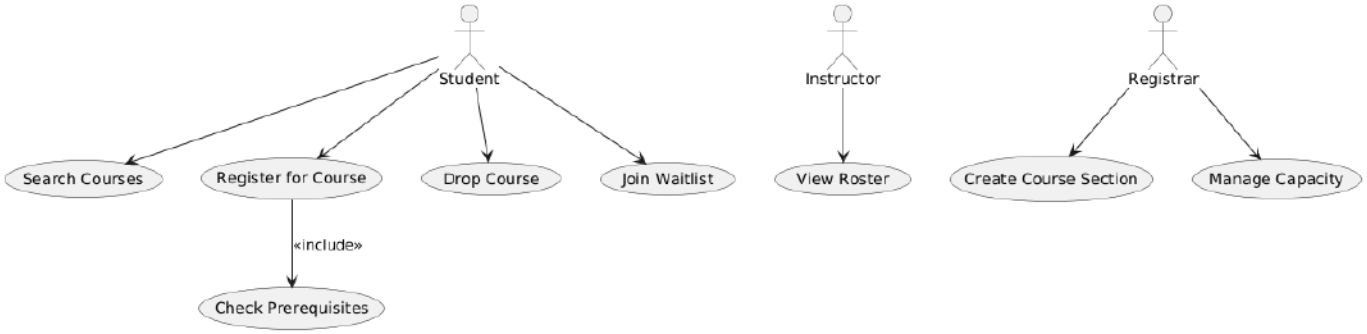

Created
paymentConfirmed
Paid
restaurantAccept
Preparing
readyForPickup
Assigned
riderPickedUp
OutForDelivery
delivered
Delivered

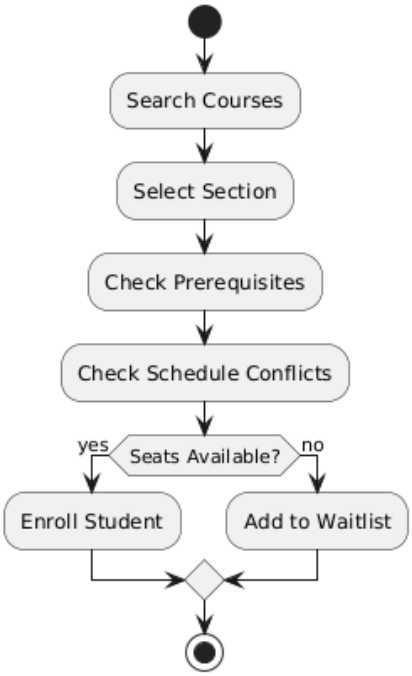

MobileApp
WebFrontend
OrderService
PaymentService
RiderService
DB
NotificationService

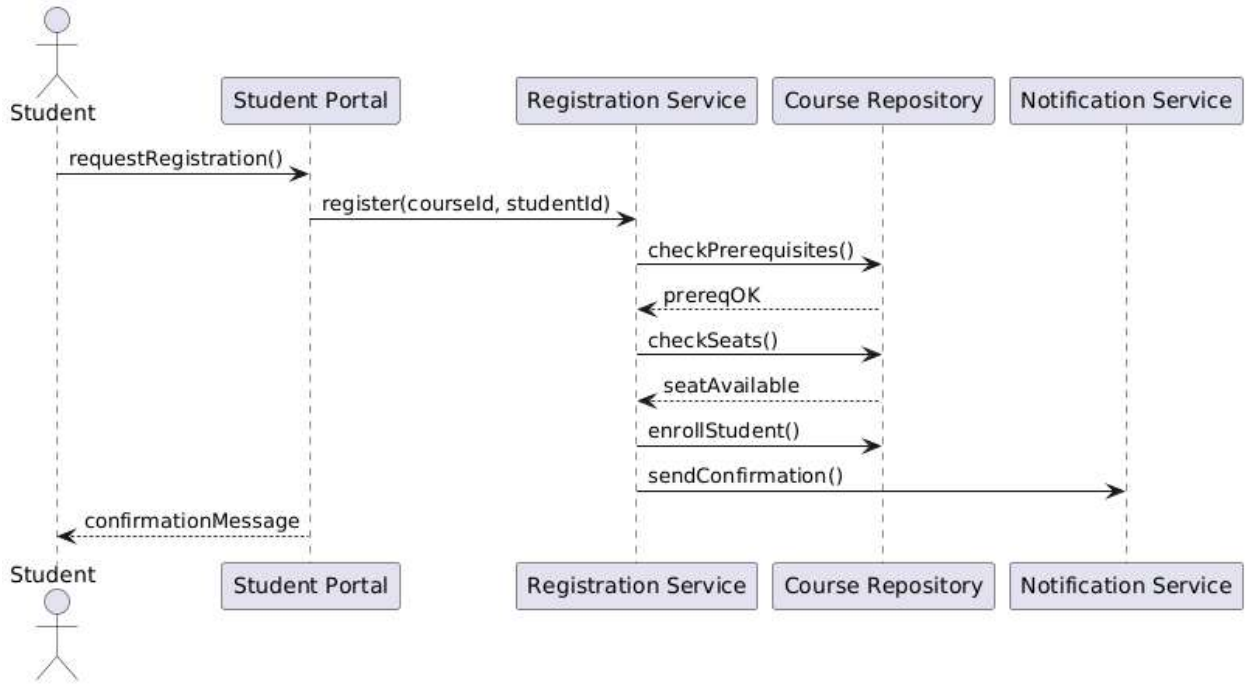

MobileDevice
MobileApp
HTTP
WebServer
OrderService
WebFrontend
Google Remote Procedure Call
API
gRPC
JDBC
RiderServer
RiderService
DBCluster
DB
PaymentGateway

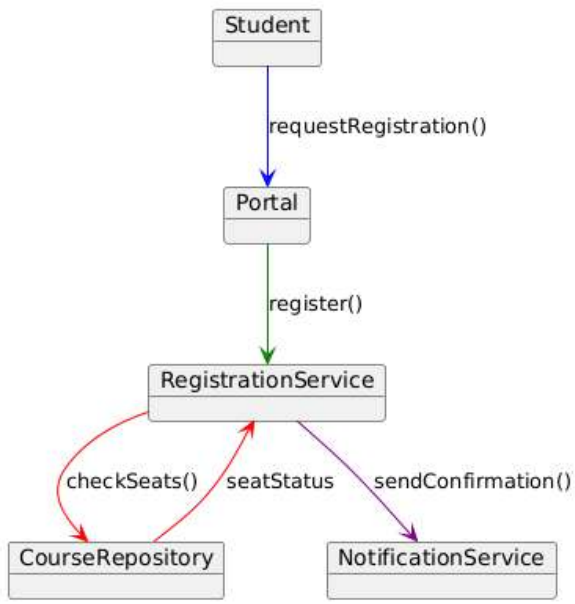
# SAMPLE 2

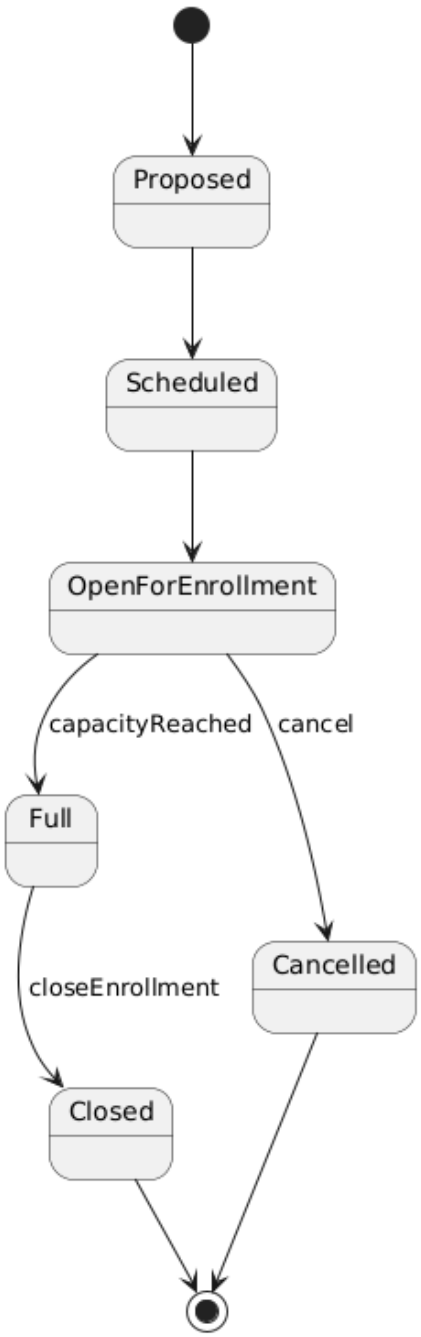
UniReg is a university's registration system enabling students to enroll in courses, a registrar to manage schedules and capacity, instructors to view rosters, and automated waitlists to handle full courses and seat openings. The system enforces prerequisites and co-requisites, supports course splitting/sections, and notifies students/instructors of changes.

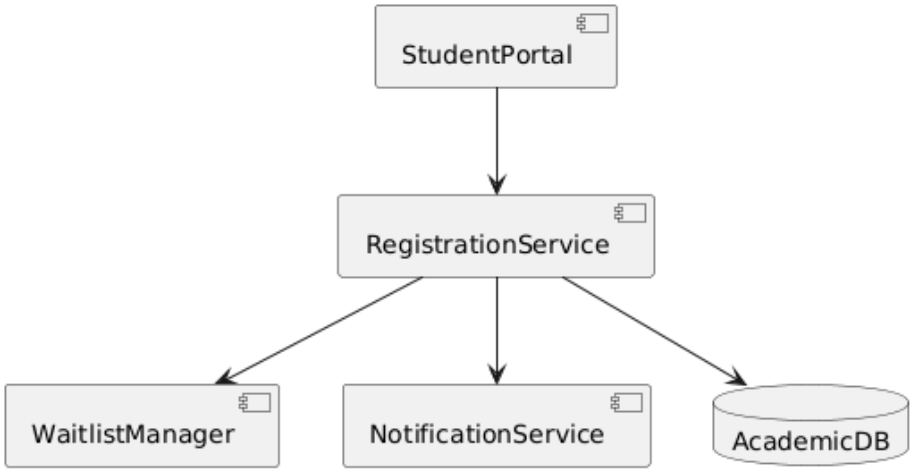


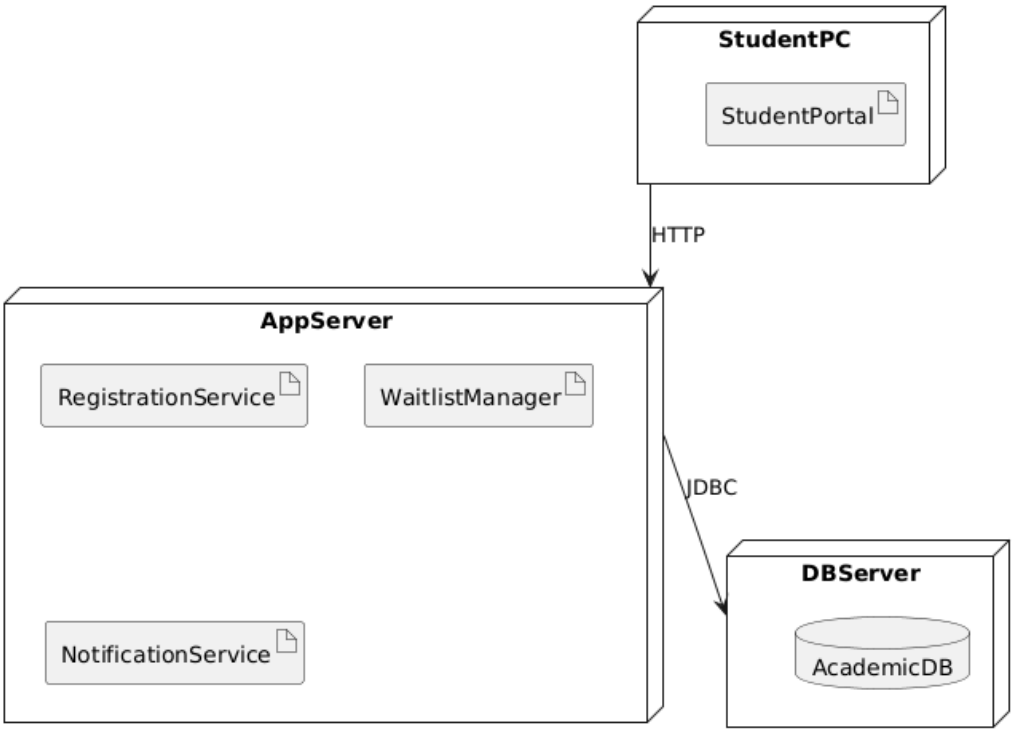




Student
Student Portal
Registration Service
Course Repository
Notification Service
requestRegistration()
register(courseId, studentId)
checkPrerequisites()
prereqOK
checkSeats()
seatAvailable
enrollStudent()
sendConfirmation()
confirmationMessage
Student
Student Portal
Registration Service
Course Repository
Notification Service

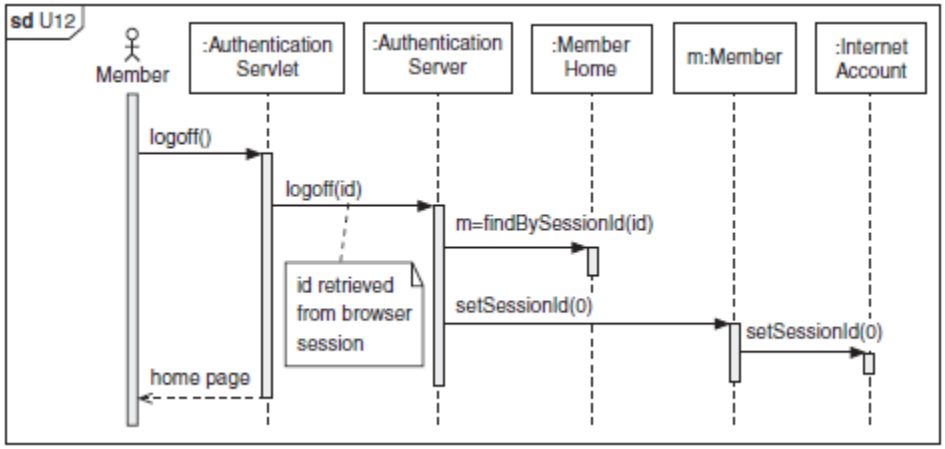

Student
requestRegistration()
Portal
register()
RegistrationService
checkSeats()
seatStatus
sendConfirmation()
CourseRepository
NotificationService


Proposed
Scheduled
OpenForEnrollment
capacityReached
cancel
Full
Cancelled
closeEnrollment
Closed


StudentPortal
RegistrationService
WaitlistManager
NotificationService
AcademicDB


StudentPC
StudentPortal
HTTP
AppServer
RegistrationService
WaitlistManager
JDBC
DBServer
NotificationService
AcademicDB

# SAMPLE 1




Analysis of the diagram with respect to object-oriented paradigm and conceptual modeling shows:

The sequence diagram reflects strong encapsulation. The Authentication Servlet acts as a controller, coordinating interactions, while responsibilities are distributed logically among domain objects.

**Requirements:**

- **Functional:**
  - The system shall allow a member to log off through the authentication interface.
  - The system shall retrieve the session ID from the browser session.
  - The system shall locate the member object by session ID using the Member Home repository/service.
  - The system shall invalidate/reset the session ID in the Member object.
  - The system shall invalidate/reset the session ID in all associated Internet Account objects.
  - The system shall redirect the member to the home page after a successful logoff.
  - The system shall delegate responsibilities properly:
  - The Authentication Servlet handles request forwarding.
  - The Authentication Server performs session validation and logoff logic.
  - The Member Home finds members by session ID.
  - The Member and Internet Account update their own session state.

- **Non-Functional:**
  - Security
    - Session IDs must be securely retrieved, transmitted, and invalidated to prevent reuse or hijacking.
    - Only authenticated/authorized users should be able to invoke logoff.
  - Reliability
    - The logoff process must consistently invalidate all active sessions for the member and related accounts.

        - The system must ensure no dangling/active session remains after logoff.
    - Performance
        - The logoff operation (session retrieval, invalidation, redirection) should complete quickly (near real-time).
    - Consistency
        - All linked objects (Member, Internet Account) must have a synchronized session state after logoff.
    - Scalability
        - The system should handle logoff requests from many users concurrently without degradation.
    - Usability
        - After logoff, the user should clearly be returned to the home page, confirming the action.
    - Maintainability/Extensibility
        - Session management logic should be centralized enough to allow easy modifications (e.g., adding multi-factor or distributed session handling later).

**<u>Object Identification:</u>**

- **Actor (External User):** Member (human user interacting with the system).
- **Boundary Objects (interfaces):** Authentication Servlet – entry point for requests.
- **Control Objects (logic/processing):** Authentication Server – handles logoff request, manages sessions.
- **Entity Objects (domain data):** Member Home – acts like a service/repository to retrieve Member by Session ID.
- Member – represents the logged-in person, has attributes like sessionId.
- Internet Account – represents associated accounts that also maintain session info.

**<u>Relationship(s):</u>**

- Member Home and Member have Association (home manages members, returns the correct one).
- Member and Internet Account have Association/Aggregation (a member has one or more internet accounts, which also need session management).
- Authentication Servlet depends on Authentication Server, Authentication Server depends on Member Home, Member Home depends on Member, Member depends on Internet Account.
- User sends logoff() request.
    - Servlet passes request to Server (service delegation).
    - Server queries repository to find Member by session ID (lookup).

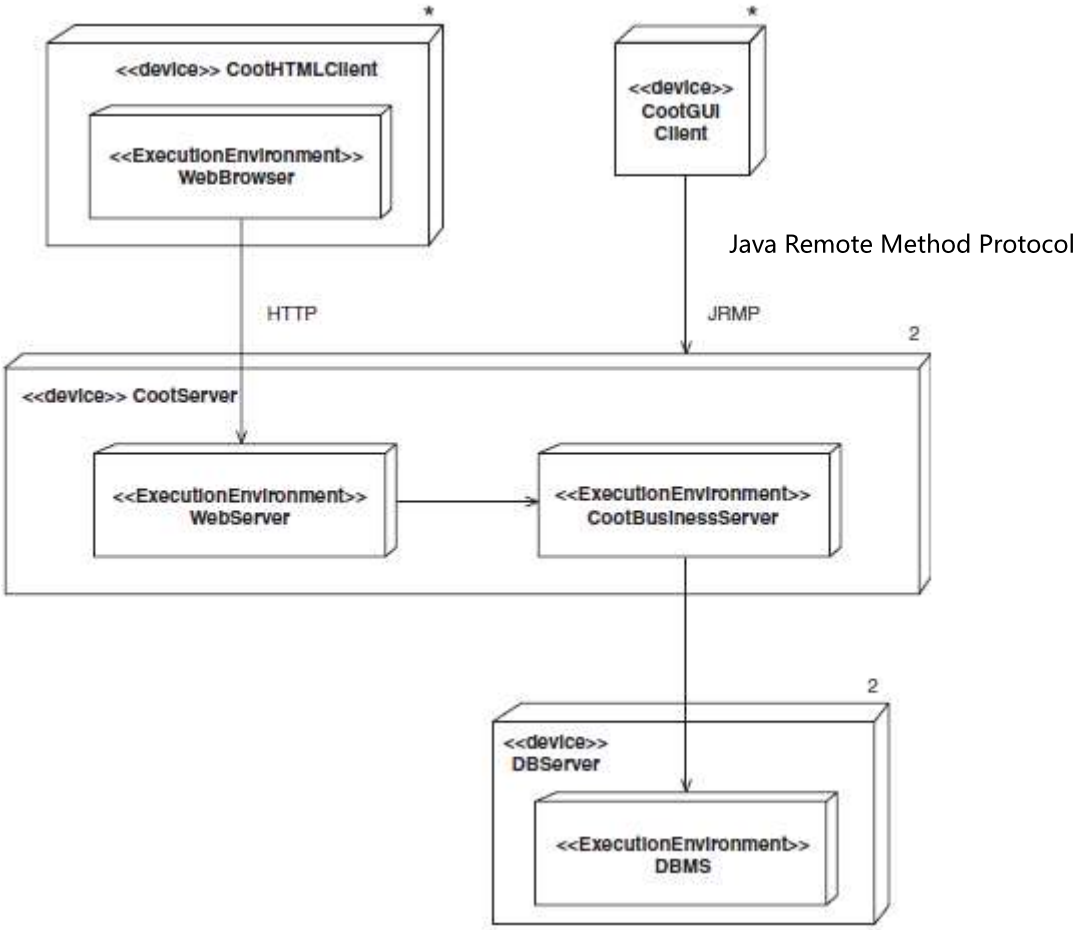
So,

- Member — (1..*) → Internet Account (aggregation: a member can have multiple accounts).
- Authentication Server — (1) → Member Home (uses it to look up members).
- Member Home — (1..*) → Member (repository association).
- Authentication Servlet — (1) → Authentication Server (controller delegation).

**Conceptual Class/Object Candidates (with Attributes/Responsibilities)**

- **Member**
  - Attributes: memberId, name, sessionId, accountId
  - Responsibilities: manage session ID, logoff.
- **Internet Account**
  - Attributes: accountId, sessionId, status
  - Responsibilities: update session state when member logs off.
- **Authentication Server**
  - Attributes: (none directly — more service-based)
  - Responsibilities: validate session, handle logoff.
- **Authentication Servlet**
  - Responsibilities: receive requests, forward to server.
- **Member Home (Repository/Manager)**
  - Responsibilities: find Member objects by sessionId.

## SAMPLE 2



Analysis of the diagram with respect to object-oriented paradigm and conceptual modeling shows:

Each device encapsulates its execution environment in such a way that internal details of each execution environment are hidden behind interfaces (HTTP, JRMP, DBMS access).

**Requirements:**

- **Functional:**
  - The system shall support access via web clients (HTTP).
  - The system shall support access via GUI clients (JRMP).
  - The system shall provide a WebServer for handling HTTP requests.
  - The system shall provide a BusinessServer for business logic execution.
  - The system shall maintain persistent data in a DBMS.
  - The system shall allow the BusinessServer to communicate with DBMS for data storage and retrieval.
  - The system shall support multiple clients simultaneously.
  - The system shall support two DB servers for redundancy/failover.

- **Non-Functional:**
  - Performance
    - Requests from multiple clients must be handled efficiently.
    - DB queries must be executed within acceptable response time.
  - Scalability
    - The system must allow many clients (*) to connect concurrently.
    - Additional DB servers (2..n) can be deployed for load balancing.
  - Reliability / Availability
    - Two DB servers ensure fault tolerance and high availability.
  - Security
    - HTTP and JRMP communications must be secured (e.g., HTTPS, encrypted RMI).
    - Authentication/authorization required for client access.
  - Maintainability
    - Separation of Web, Business, and DB layers ensures easier updates.
  - Portability
    - Clients should work on different platforms (Web browser independence, GUI client on multiple OS).
  - Consistency
    - DBMS must ensure transactional consistency across multiple servers.

**Object Identification:**

- **Devices (hardware nodes) –** CootHTMLClient, CootGUIClient, CootServer, DBServer.
- **Execution Environments (software nodes) –** WebBrowser, WebServer, CootBusinessServer, DBMS.
- **Connectors/Protocols –** HTTP, JRMP, DBMS communication channel.

**Relationship(s):**

- **Clients – Server**
  - Web client uses HTTP to talk to WebServer.
  - GUI client uses JRMP to talk to BusinessServer.
- **Server – Business Logic**
  - WebServer delegates to CootBusinessServer.
- **Server – Database**
  - CootBusinessServer communicates with DBMS.
- **Multiplicity**
  - Many clients can connect simultaneously (*).
  - Two DB servers are available (2) → redundancy/fault tolerance.

**Conceptual Class/Object Candidates (with Attributes/Responsibilities)**

- **CootHTMLClient**
  - Attributes: browser session, HTTP connection.
  - Responsibilities: send/receive HTTP requests.
- **CootGUIClient**
  - Attributes: GUI state, JRMP connection.
  - Responsibilities: send/receive JRMP requests.
- **CootServer**
  - Attributes: server configuration, active connections.
  - Responsibilities: host WebServer and CootBusinessServer.
- **WebServer**
  - Responsibilities: handle HTTP requests, forward to business logic.
- **CootBusinessServer**
  - Responsibilities: core business logic, database access orchestration.
- **DBServer (DBMS)**
  - Attributes: schema, data tables.
  - Responsibilities: manage persistent data, handle queries/updates.